A screenshot or a sheet without your student ID in it is worth zero marks

Homework 1

Blockchain Basics and Platforms • Inha University in Tashkent • Fall 2026

Assigned	04.10.2026	**Instructor**	Yusupov Jalolliddin
Due	18.10.2026, 23:59	**Weight**	100 points = 5%
Submit	One ZIP on eClass	**Work**	Individual

What you submit

One ZIP file, containing exactly this:

```
SOC4190_HW1_<StudentID>_<Surname>.zip
├── report.pdf       ← the answers, the screenshots, the photos of your handwriting
└── src/
    ├── blockhash.py  ← Part 3, typed in by you
    └── header.hex    ← Part 3, the header of your own block
```

The report is **one PDF**. Write it in Word (using the template provided) or LibreOffice and export to **PDF**. Number your answers exactly as this handout numbers the questions — 1.1, 2.3, 4.2 — so we can find them.

Your own numbers [INFO]

Almost every task uses numbers taken from **your student ID**, so your answers are different from everyone else's. Work them out once, write them at the top of your report, and use them everywhere.

Name	How to get it	Example: ID U2310123
Block height **H**	the last five digits of your ID, read as a number (if that is below 10, add 10,000)	10123 → block 10,123
Leaves **L1 L2 L3 L4**	the last four digits of your ID, each one plus 10	0, 1, 2, 3 → 10, 11, 12, 13
Key **K**	the last digit of your ID, plus 3	3 → K = 6
Amount **A**	the second-to-last digit of your ID, times 5, plus 10	2 → A = 20

The rules for screenshots

Many of the marks are for screenshots. A screenshot only counts if **all three** of these are true:

1. It shows **your own** work. In a terminal, the prompt and the command you typed must be visible, and your student ID must appear in the command or in the input. On the explorer, the block height in the picture must be **your** H.
2. The **whole result** is visible, not a cropped corner. For the explorer, include the address bar of the browser.
3. It is **readable**. A photograph of your monitor taken with a phone is not readable. Use the screenshot tool of your computer.

The rules for handwriting

The exercises marked ✍ are done **on paper, by hand**. Write your name and student ID at the top of every sheet, photograph or scan the sheets, and put the pictures in the report. Typed tables, typed trees and typed calculations score zero for these exercises. **Show the arithmetic**, not only the result: we mark the steps.

Part 1 — The ledger, by hand [15 points]

From Lectures 1 and 2. No computer is needed for this part.

1.1 ✍ — From records to balances [6]

Four friends each start the week with 100 (thousand so‘m).
Their shared notebook for the week is below. Line 6 uses your own amount **A**.

```
1  Mon  Aziz    → Dilnoza · 40
2  Mon  Dilnoza → Bekzod  · 25
3  Tue  Bekzod  → Malika  · 60
4  Wed  Malika  → Aziz    · 15
5  Thu  Dilnoza → Malika  · 30
6  Fri  Aziz    → Bekzod  · A
```

Hand in: by hand, a table with the balance of each friend after every line; the four final balances; and their sum. Then one sentence: why is the sum the same as at the start of the week?

1.2 — The same coin, twice [5]

On Saturday Bekzod has exactly 50. He writes two lines at the same moment,
in two different copies of the notebook: “Bekzod → Aziz · 50” and “Bekzod → Malika · 50”.

Hand in: three short answers. (a) What is this problem called? (b) How does a bank make it impossible? (c) The friends have no bank, and everyone keeps a copy of the notebook. Which two things must all copies agree on, so that only one of the two lines survives?

1.3 — Where are the copies, and who is in charge? [4]

For each system, say whether it is **centralised or decentralised**, and whether it is **distributed or not**, with one reason each.

a	A bank keeps its ledger on servers in three cities. All three belong to the bank.
b	One teacher keeps the attendance list in a paper notebook.
c	Bitcoin: thousands of nodes, run by different people, each with the full ledger.
d	Twenty students each keep a copy of a shared file, but only the class leader is allowed to change it.

Hand in: a four-row table with your two answers and the reasons.

Part 2 — Your block on the explorer [20 points]

From Lecture 3. You open one real Bitcoin block — block number **H** (read page 1 [INFO]) — and read it the way we read block 969,431 in the lecture. Use **mempool.space**.

2.1 — Open your block [5]

1. Open mempool.space and type your height **H** into the search box.
2. Press **Details** to see the header fields.
3. Copy the six fields of the header into a table, as in the lecture.

Field	Value in your block
Version	
Previous block hash	(the last 12 characters are enough)
Merkle root	(the last 12 characters are enough)
Timestamp	(the date and time the explorer shows)
Bits	
Nonce	

Hand in: a screenshot of the block page with the height visible, a screenshot of the Details section, and the filled table.
Add two numbers: how many transactions your block holds, and its size in bytes.

2.2 ✍ — From seconds to a date [5]

The explorer shows the timestamp as a date. The block itself stores one number: seconds since 1 January 1970 (Unix time). Get that number for your block with this address in your browser, replacing the hash with your block's hash:

> https://mempool.space/api/block/<the hash of your block>

Find "timestamp" in the answer.

Hand in: by hand, the conversion of your timestamp into a date and a time (UTC), with every division shown, as on the slide “When was block 0 made?”. Your blocks are from 2009 or 2010. Say whether your result agrees with the explorer.

2.3 ✍ — Draw the chain [6]

Open the block before yours (H − 1) and the block after yours (H + 1).

Hand in: by hand, the three blocks drawn side by side. In each: the height, the time, the last 12 characters of “previous block hash”, and the last 12 characters of the block's own hash. Draw the arrows. Under the drawing, write which two pairs of values must be equal — and whether they are.

2.4 — Measure the rhythm [4]

Hand in: three numbers and one sentence. (a) The seconds between block H − 1 and block H. (b) The seconds between block H and block H + 1. (c) The average time per block from block 0 to your block: (your timestamp − 1,231,006,505) ÷ H, in minutes.
(d) Bitcoin aims at ten minutes. Why are (a) and (b) not ten minutes?

Part 3 — Fingerprints on your machine [25 points]

From Lecture 4. You need a terminal: Linux, macOS, or Windows with WSL or Git Bash.
On Linux write sha256sum where this handout writes shasum -a 256.

3.1 — One character changes everything [7]

Compute SHA-256 of your student ID.
Then change its **last character** to any other character and compute again. Use printf, not echo.

```
printf "U2310123" | shasum -a 256
printf "U2310124" | shasum -a 256
```

Hand in: one screenshot with both commands and both results. Then ✍ by hand: write the two hashes one under the other and mark every position where the two characters are equal.
How many of 64 are equal? About how many would you expect by pure chance, and why?

3.2 — Same size, whatever goes in [4]

Compute SHA-256 of three things: nothing at all (printf ""), your full name, and any file on your computer larger than one megabyte (shasum -a 256 <file>).

Hand in: a screenshot of the three commands and results, the size of the file in bytes, and one sentence naming the promise of a hash function that this experiment shows.

3.3 — Compute the hash of your own block [10]

On the Details section of your block there is a line **Block Header Hex**.
It opens 160 hexadecimal characters: the 80 bytes of the header. Save them in a file called `header.hex`. Then type in this program — do not paste it — and save it as `blockhash.py`:

```
import hashlib, sys
header = bytes.fromhex(open(sys.argv[1]).read().strip())
once  = hashlib.sha256(header).digest()
twice = hashlib.sha256(once).digest()
print(len(header), "bytes  block hash:", twice[::-1].hex())
```

Requirement	Points
`python3 blockhash.py header.hex` prints **80 bytes** and a hash; the command and the result are in a screenshot.	3
The printed hash is equal, character by character, to the hash of your block on the explorer. Show both in the report and mark that they agree.	3
Make a copy of `header.hex`, change its **last character** to a different one, and run the program on the copy. Screenshot the result.	2
Two sentences: what happened to the zeros at the start of the hash, and which promise of a hash function explains it?	2

Hand in: the three screenshots, the two sentences, and the files `blockhash.py` and `header.hex` in `src/`.

3.4 ✍ — Twins of a toy hash [4]

The toy hash from the lecture: **add the digits of a number and keep the last digit of the sum**.
Example: toy(2026) = 2 + 0 + 2 + 6 = 10 → 0.

Hand in: by hand: (a) the toy hash of the seven digits of your student ID, read as one number; (b) two other seven-digit numbers with the same toy hash; (c) two sentences: why are twins so easy to find here, and why has nobody found one for SHA-256?

Part 4 — Merkle trees, by hand [25 points]

From Lecture 4. This whole part is **handwritten**. SHA-256 is too big for paper, so we use the toy rule (simpler hash function) of the lecture for a pair of numbers:

> **T(a, b) = (31 × a + b) mod 97**
> Example: T(65, 66) = 2081 mod 97 = 44, because 2081 = 21 × 97 + 44.

Your four leaves are **L1, L2, L3, L4** from the table “Your own numbers”. For the example ID they are 10, 11, 12, 13, and the tree is: T(10, 11) = 30, T(12, 13) = 94, root T(30, 94) = 54.

4.1 ✍ — Build your tree [7]

Hand in: the tree drawn by hand, with the four leaves at the bottom, the two middle values and the root, and the full arithmetic for each of the three T(·,·): the multiplication, the sum, and the division by 97 with its remainder.

4.2 ✍ — Change one leaf [5]

Add 1 to your second leaf, L2, and leave the others as they are.

Hand in: the new tree. Circle every value that changed.
One sentence: which values did **not** change, and why?

4.3 ✍ — A proof for one leaf [6]

Go back to your original tree from 4.1. A friend knows only the **root**.
You want to convince the friend that **L3** is in the tree, without showing the other leaves.

Hand in: (a) the two values you must give the friend, and where each one sits in the tree; (b) the two calculations the friend performs, written out; (c) what the friend compares at the end.

4.4 ✍ — Five leaves [3]

Add a fifth leaf, **L5 = 20**, to your original four. Use Bitcoin's rule (told you so xD):
when a level has an odd number of values, the last one is paired with a copy of itself.

Hand in: the tree, level by level, down to the new root.

4.5 ✍ — How long is a proof? [4]

Hand in: by hand, with the powers of two shown: the number of hashes in a Merkle proof, and its size in bytes (32 bytes per hash), for a block with (a) 1,000 transactions, (b) 6,816 transactions, (c) 1,000 + H transactions, where H is your block height.

Part 5 — Keys and signatures [15 points]

From Lecture 4.

5.1 ✍ — Caesar [5]

Hand in: by hand, with the alphabet written out: (a) your surname encrypted with your key **K**;
(b) the message AHZORLUA decrypted with key 7;
(c) one sentence: why is this kind of cryptography called symmetric?

5.2 — Which key? [4]

	Situation	Private or public key?
a	Aziz signs the transaction “Aziz → Timur · 5”.	
b	A node checks that the signature on that transaction is valid.	
c	Timur wants to check the signature himself, on his own laptop.	
d	Aziz must never show this key to anyone.	

Hand in: the table, filled in, with whose key it is in each row.

5.3 — Three short answers [6]

Hand in: two or three sentences each. (a) Timur copies the signature from “Aziz → Timur · 5” and attaches it to a new message, “Aziz → Timur · 500”. What does a node find when it verifies, and why? (b) “The transaction is signed, so nobody can read it.” What is wrong with this sentence?
(c) Aziz loses his private key.
Who can give him a new one for the same coins, and what happens to the coins?

Submission checklist

(strictly follow the rules)

- The ZIP is named SOC4190_HW1_<StudentID>_<Surname>.zip.
- report.pdf is inside it, and opens.
- Your four numbers — H, the leaves, K and A — are written at the top of the report.
- Every screenshot shows your own block or your own student ID.
- Every ✍ exercise is a photograph of handwriting, with your name and ID on the sheet.
- src/ contains blockhash.py and header.hex, and the program runs.